

ACRS

Design and Evaluation of a Risk-Adaptive Cyber Containment Model Based on Threat Severity and Asset Criticality

ACRS: Adaptive Cyber Response System · Stops the attack, not the hospital

37 → 20

machines offline

7 → 2

critical outages

36 = 36

attacker stopped (of
41)

Research & development: Hassan Abdullah Alyenbawi

All data in this paper is simulated. No real network was touched.

All data in this paper is simulated. No real network was touched.

Key result

Problem or warning

What ACRS adds

Abstract

When a machine in an organization is hacked, most automated response tools fully isolate it from the network. That stops the attacker, but **it can also stop the login server every employee depends on, or a medical device a patient depends on**. This research proposes a system called ACRS that **picks the lightest response that still stops the attacker**, based on three things: how dangerous the attack is, how sure we are, and how important the machine is. We compared ACRS with the usual way, full isolation of any machine whose danger score passes 60, in a simulated test of 42 incidents with exactly the same inputs. Result: **20 machines stopped instead of 37, 2 important systems stopped instead of 7, and the attacker stopped in 36 of 41 attacks with both approaches**. No fixed number from 50 to 100 achieved both results. In a second, 8-round experiment where the attacker changes tactics, **each new trick worked only once** after ACRS learned from the engineer's response. **The cost is human involvement at specific points**.

Keywords: incident response, containment, automated response, asset criticality, ransomware

1 Introduction and the problem

Cyber attacks are getting faster and more damaging. In Verizon's yearly reports, the share of breaches involving ransomware rose **from 25% in 2022 to 48% in 2026** [1]. CrowdStrike reports that the average time an attacker needs to move from the first machine to others was **29 minutes** in 2025 [2]. The global average cost of a breach is **\$4.99 million** [3].

Because attackers are fast, organizations rely on automated response tools that act in seconds instead of waiting for a person. The most common response in these tools is simple: fully isolate the infected machine from the network.

The problem is that this is one response for every machine. Isolating an employee's laptop hurts no one, **but isolating the login server stops every employee, and isolating a medical device can hurt a patient**. Incident response guidance already says a containment decision should consider its impact on services [4]. So the question is not whether to stop the attacker, but **what is the lightest response that still stops them?**

2 Research question and hypothesis

Research question: Can a system that weighs how dangerous the attack is and how important the machine is stop fewer machines than full isolation, and still stop the attacker just as often?

Hypothesis: a system that considers how important the machine is will stop fewer machines and fewer important systems, without stopping the attacker any less often.

Null hypothesis: there is no difference in machines stopped, or ACRS stops the attacker in fewer attacks.

Objectives:

1. Build ACRS as a decision system with clear rules that can be checked, with a written reason for every decision.
2. Test it against full isolation on the same simulated attacks.
3. Measure machines stopped, attackers stopped, speed, and the human effort needed.

3 How ACRS works

Every alert in ACRS goes through eight stages. The experiment changes only stage five, the decision:

1. Detect: 12 rules reading from 8 kinds of sensors.
2. Connect: related events within 5 minutes are grouped into one incident.
3. Score danger: a score from 0 to 100 from 6 factors.
4. Machine importance: is it core, important or ordinary? Does it have a backup? Is it a safety device?
5. Decide: 9 ordered rules, no AI.
6. Contain: carry out the command within 7, 15 or 90 seconds depending on the machine.
7. Check: 9 checks confirm the containment worked.
8. Return: the machine comes back to work in stages.

Machines in ACRS have three levels: core (Tier-0) such as login and certificate servers, important (Tier-1) such as databases and medical devices, and ordinary (Tier-2) such as employee laptops.

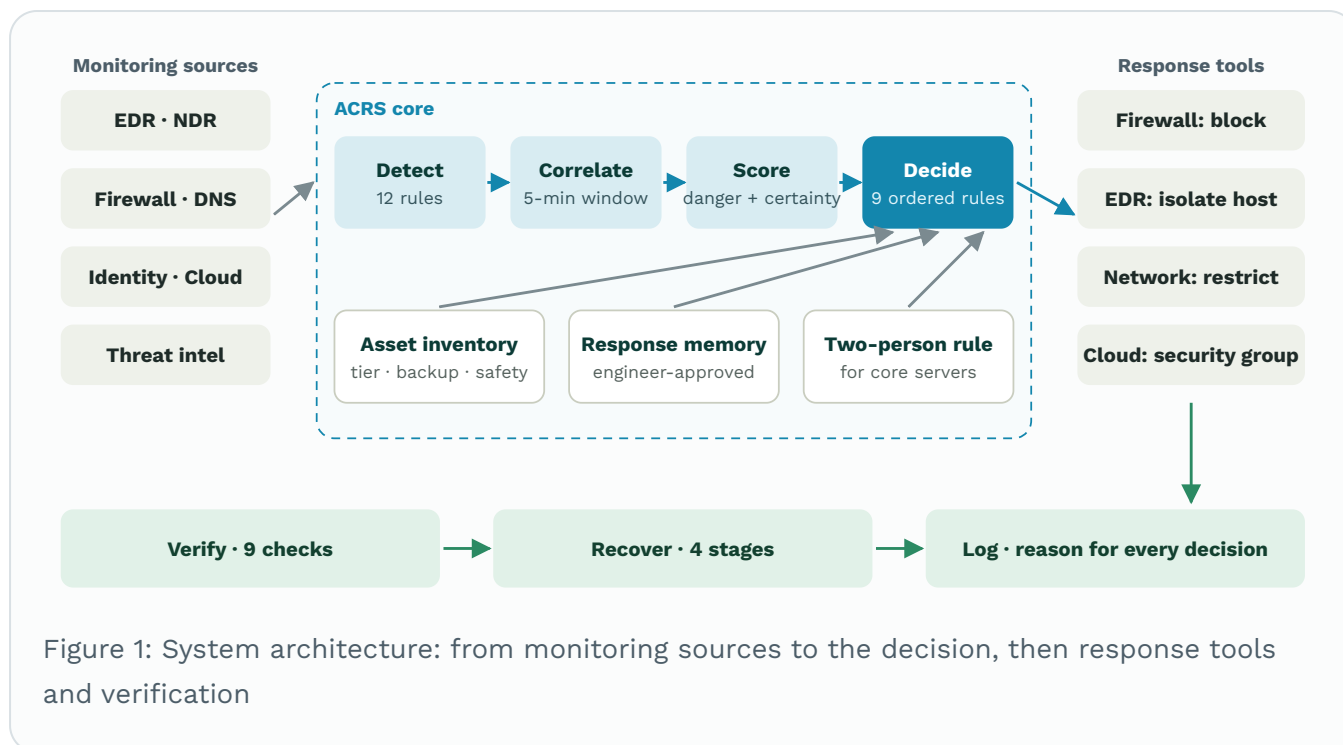


Figure 1: System architecture: from monitoring sources to the decision, then response tools and verification

The nine decision rules, in order. The first rule that applies decides:

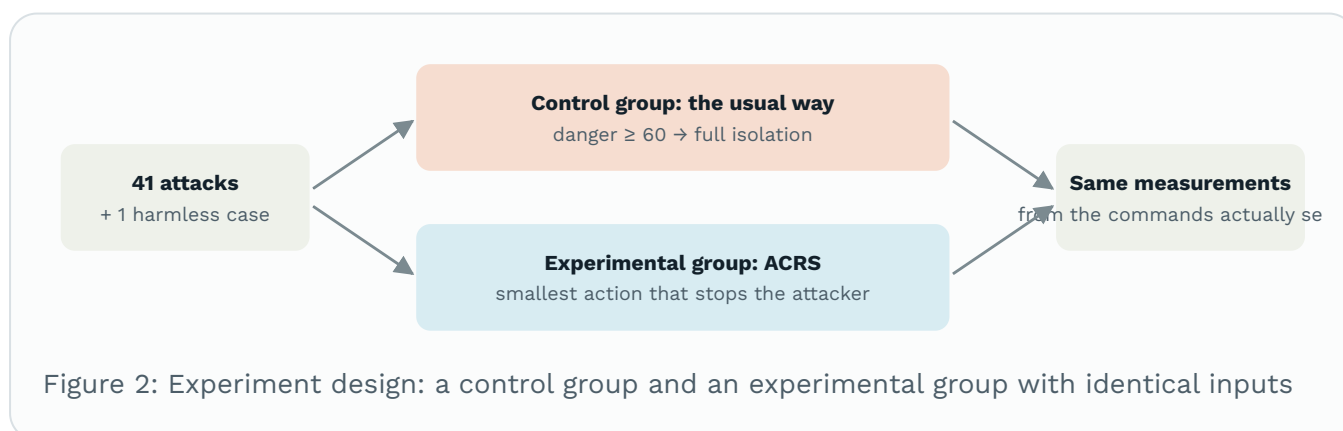
#	If...	then ACRS...
1	the machine is unknown	blocks its address at the network edge only
2	the machine must not be switched off (medical or industrial)	blocks the attacker on the network and leaves the machine running
3	the attack is ransomware	acts at once: core is restricted, important is isolated while its backup runs, ordinary is fully isolated
4	certainty is below 60% (one sensor is capped at 55%)	does not isolate on its own and waits for a person
5	danger is below 60	watches only
6	the machine is in an approved maintenance window	restricts and waits for confirmation
7	danger is above the machine's limit (core 95, important 90, ordinary 85)	core: immediate restriction, two people for full isolation. Important: isolated while the backup runs. Ordinary: full isolation

#	If...	then ACRS...
8	danger is medium	restricts on its own; a stronger step needs a person. Core: watch, and two people
9	5 or more automatic containments happened within 10 minutes	pauses automation so it does not isolate the whole organization

What ACRS adds: automatic isolation already exists in commercial security tools, and we do not claim to have invented it. What this research adds is **a clear decision model that sets how strong the response is from danger, certainty and machine importance**, and a controlled test comparing it with full isolation.

4 How we tested it

The experiment has two arms: every incident goes through both approaches with exactly the same information, meaning the same detection, the same danger score and the same certainty. **The only thing that changes is the decision.**



The usual way (baseline): if danger is 60 or more, or the attack is fast ransomware, the machine is fully isolated at once. Otherwise it is watched. We chose 60 because it is the same minimum ACRS uses, so the comparison is fair.

Type	Variable
Independent (what we changed)	The response approach: the usual way or ACRS
Dependent (what we measured)	Machines stopped, important systems stopped, medical devices switched off, core servers isolated with no person, the harmless case, attacker stopped, paths cut, response speed, human effort
Controlled	Same scenarios, same detection, same danger and certainty scoring, same machine list, same way of turning a decision into commands, same way of counting

Data: 40 attack scenarios of 8 types, producing 42 incidents: 41 attacks and one harmless case (legitimate maintenance that looks like an attack). There are 20 machines: 4 core, 10 important including an MRI controller, and 6 ordinary. Danger ranged from 40 to 99, and each incident was seen by 1 to 4 sensors.

Attack type	Incidents
Ransomware	7
Attacker command channel	6
Identity theft	5
Initial access	5
Edge cases	5
Cloud & containers	5
Web apps & APIs	5
Insider data theft	4

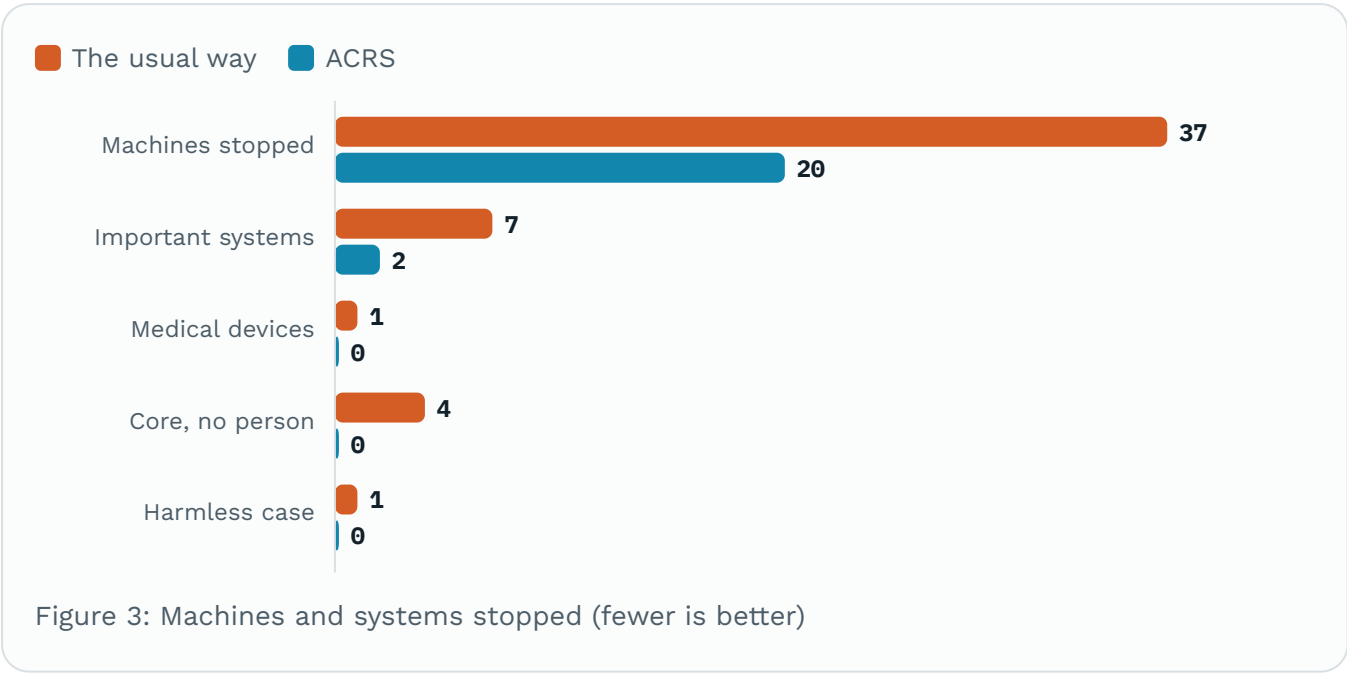
How we counted: not from the decision's name, but from the commands each approach actually issued. For example, we only count the attacker as stopped if a command cut their outside connection.

Reproducibility: the system has no randomness, so re-running the experiment **gives exactly the same numbers**. The rules file fingerprint is 87893ca0, and 6 automated tests check on every run that the results have not changed.

5 Results

Table 1: Main results on 42 incidents

What we measured	The usual way	ACRS
Machines stopped	37	20
Important systems stopped	7	2
Attacker stopped (of 41 attacks)	36	36
Paths to important systems cut	36	30
Medical devices switched off	1	0
Core servers isolated with no person	4	0
Harmless case stopped by mistake	1	0



ACRS stopped the attacker in the same number of attacks (36 of 41), but stopped 20 machines instead of 37, **which is 46% fewer**. Important systems stopped **fell from 7 to 2**.

Is it enough to raise the isolation number in the usual way? We tried every number from 50 to 100:

Table 2: The usual way with different isolation numbers

Isolation number	Important systems stopped	Attacker stopped
50	7	38
55	7	37
60	7	36
65	7	34
70	7	33
75 and 80	5	32
85	5	24
90	2	18
95	2	10
100	1	7
ACRS	2	36

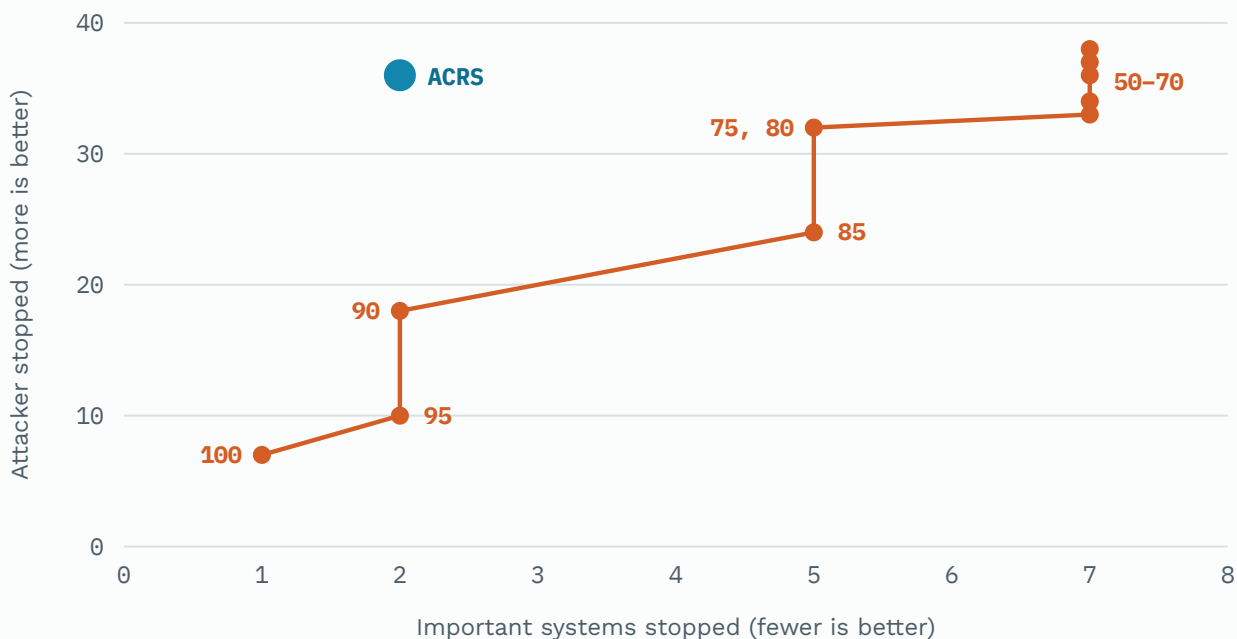


Figure 4: Each orange dot is the usual way with a different isolation number

To get down to 2 important systems stopped, the usual way needs the number 90, and then **it stops the attacker in only 18 attacks**. **Only ACRS achieved 2 and 36 together**.

ACRS decisions across the 42 incidents: 17 restricted while working, 9 smart isolation, 7 full isolation, 4 immediate restriction, 4 watch, and 1 blocked on the network only. The usual way fully isolated 37 and watched 5.

Speed: across 5 full runs, time to contain was **0.29 to 0.34 seconds for both approaches**, and no containment missed its deadline in 365 actions. The decision itself takes about 2 microseconds in ACRS and 0.8 in the usual way, so the time difference is negligible.

6 Second experiment: failure is never final

When ACRS does not stop an attack automatically, an engineer handles it by hand. This experiment asks: **if ACRS saves the engineer's response and reuses it on similar attacks, how many attacks get through when the same attacks come back, even if the attacker changes tactics?**

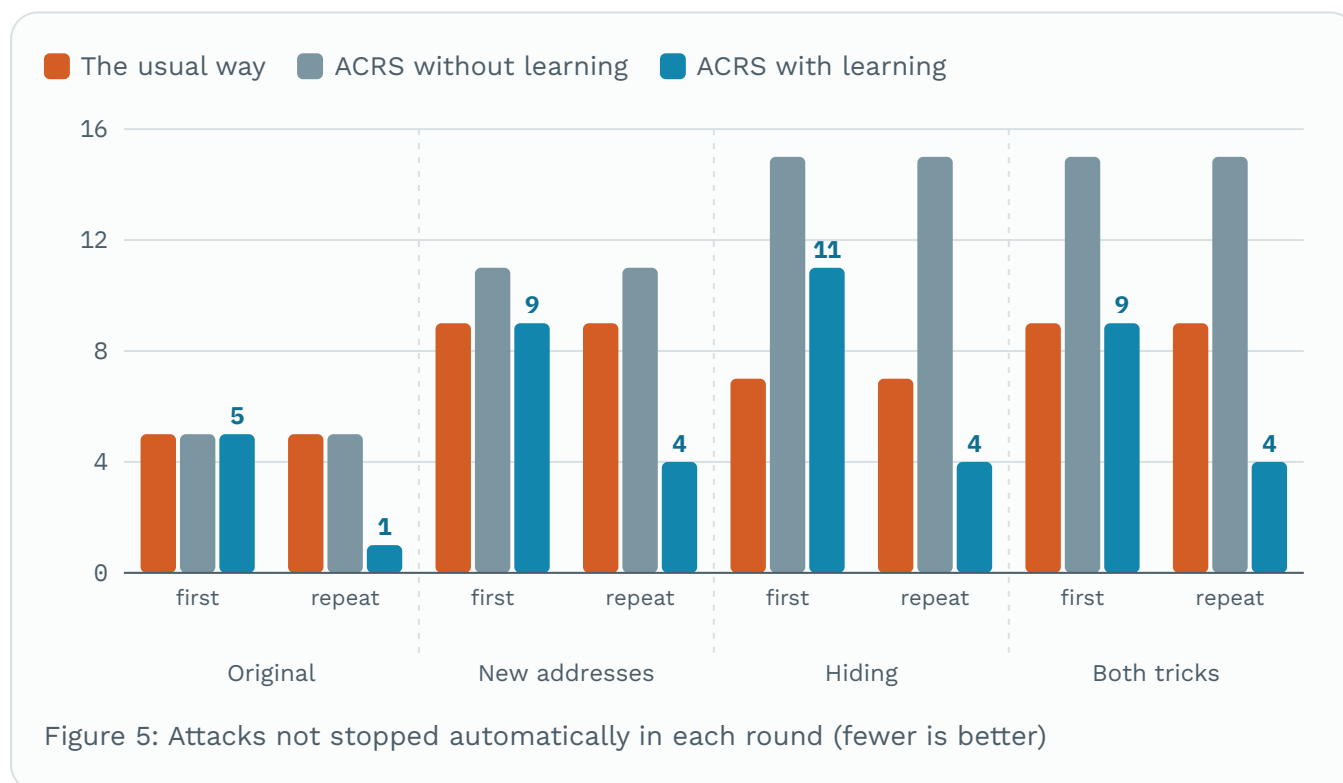
Method: 8 rounds, each with the same 41 attacks, and the attacker changes tactics every two rounds: the original attacks, then new addresses that threat feeds do not know, then hiding from one sensor, then both tricks together. Every version goes through the same, unmodified engines. After each round, every attack that was not stopped automatically is handled by the engineer (simulated, and assumed to identify the attack correctly), and the response is saved.

Safety limits on the learned response: it is used only when a new attack matches an approved response at 96% or more, never on a core server, and the strongest thing it does is restrict while

the machine keeps working (or block on the network for a device that must not be switched off). It never isolates a machine.

Table 3: Attacks not stopped automatically (of 41) in each round

Round	The usual way	ACRS without learning	ACRS with learning
1: original attacks	5	5	5
2: repeat	5	5	1
3: new addresses	9	11	9
4: repeat	9	11	4
5: hiding from a sensor	7	15	11
6: repeat	7	15	4
7: both tricks	9	15	9
8: repeat	9	15	4



Result: **each new trick worked only once**. In every repeat round the count fell to 4, and **all 4 are core servers** where ACRS waits for a person on purpose. So on every other machine, the attacks that got through reached zero. The learned response was never used on the harmless case or on a core server.

An important note: **without learning, the tricks raise the attacks that get past ACRS to 15**, more than the usual way (9), because ACRS does not act alone when certainty drops. Learning is what fixes this weakness when an attack returns. And **a brand-new trick is not caught the first time**: the engineer has to see it once.

The experiment is in `research/learning.py`, with 3 automated tests, and gives the same numbers on every run.

7 Discussion

Why did ACRS work? **Because it does not treat every machine the same way**. An ordinary machine is fully isolated, just like the usual way. A server with a backup is isolated while work moves to the backup, a medical device is not switched off and the attacker is blocked on the network, and a core server is never fully isolated without two people approving.

The first cost: ACRS cut fewer paths to important systems (30 instead of 36). This is on purpose: 4 core servers wait for two people to approve, a medical device was not switched off, and one alert was seen by a single sensor. But it is a real trade-off and we state it clearly.

The second cost: human effort. A person confirms a machine's return in 16 cases, 4 core servers wait for two people, and 2 cases wait for a person before any containment (the harmless case and the single-sensor alert). The usual way needs no one, but it isolates core servers with no human decision at all.

ACRS still stopped two important systems: the ERP-DB-01 database and the PKI-CA-01 certificate server. Both have no backup and faced high danger.

8 Limitations

1. The data is simulated: 40 scenarios on 20 virtual machines.
2. The impact of an outage is estimated from the machine descriptions, not measured on live services.
3. The danger weights were set by design, not tuned on real attacks.
4. **The same person wrote the scenarios and the rules, which may favour ACRS**. This is the most important limitation.
5. Time to contain was measured on simulated tools with realistic delays, not on real tools.
6. ACRS cuts fewer paths (30 instead of 36), an intended but real trade-off.
7. In the learning experiment the engineer is simulated and assumed correct, and the tricks are generated from the same attacks, not entirely new attacks.

9 Future work

- A read-only trial on real alerts: ACRS decides and logs but does not act, then the weights are tuned.
- Attack scenarios written by other people and run without changing the rules, to remove single-author bias.
- Connect one real protection tool in a test environment.
- A free basic version, with paid support for hospitals and small companies.

10 Conclusion

In this simulation, responding to match the danger **stopped 46% fewer machines** and fewer important systems (2 instead of 7), stopped the attacker just as often, and just as fast. No fixed isolation number could reach the same result. With learning from the engineer, no trick worked more than once. **The hypothesis holds in this simulation**; the next step is to test it on real data with scenarios written by someone else.

References

1. Verizon. Data Breach Investigations Report, 2022–2026. Share of breaches involving ransomware: 25%, 24%, 32% (with other extortion), 44%, 48%.
 2. CrowdStrike. 2026 Global Threat Report. Average attacker breakout time of 29 minutes (2025).
 3. IBM. Cost of a Data Breach Report 2026. Global average breach cost of \$4.99 million.
 4. NIST SP 800-61 Rev. 2 (2012) and Rev. 3 (2025). Incident response guidance.
 5. NIST SP 800-207 (2020). Zero Trust Architecture.
 6. MITRE ATT&CK, Enterprise Matrix (T1486, T1490).
 7. National Cybersecurity Authority (NCA). Essential Cybersecurity Controls (ECC).
-

© 2026 Hassan Abdullah Alyenbawi. All rights reserved.